CUDA & GPU Programming Practice Test

Part 4 - GPU Architecture, Occupancy & Applied CUDA Practice — Questions 46-60 of 60 — Answers & Explanations

46. What is a 'warp' in NVIDIA GPU architecture?

A. A software tool for debugging kernels
B. A group of typically 32 threads that execute instructions together in lockstep on an SM
C. A unit of GPU memory equal to one cache line
D. A single physical GPU core

Correct Answer: B

Explanation: A warp is the fundamental unit of thread scheduling on NVIDIA GPUs, where a fixed number of threads (commonly 32) execute the same instruction simultaneously.

47. What happens when threads within the same warp take different branches of an if-else statement (warp divergence)?

A. The GPU executes both branches serially for the affected threads, reducing parallel efficiency
B. The kernel immediately crashes
C. The GPU automatically merges the branches into one instruction with no performance cost
D. Only the first thread in the warp is allowed to execute

Correct Answer: A

Explanation: When threads diverge within a warp, the hardware must execute each distinct code path serially while masking inactive threads, reducing effective parallelism.

48. Why is it generally recommended to minimize warp divergence in performance-critical CUDA kernels?

A. Because divergence increases the number of available registers
B. Because divergence causes permanent GPU hardware damage
C. Because divergence forces serialized execution of different branches, reducing the GPU's parallel throughput
D. Because divergence only affects code running on the CPU

Correct Answer: C

Explanation: Since divergence causes different code paths within a warp to run serially rather than in parallel, minimizing it helps preserve the GPU's parallel execution efficiency.

49. What is the purpose of the __syncthreads() function in a CUDA kernel?

A. To permanently stop a subset of threads in the block
B. To synchronize all threads across every block in the grid
C. To pause the entire GPU until the CPU sends a signal
D. To synchronize all threads within a block, ensuring they reach the same point before continuing

Correct Answer: D

Explanation: __syncthreads() acts as a barrier that ensures all threads in a block wait until every thread has reached that point, which is essential when using shared memory cooperatively.

50. Why is __syncthreads() commonly needed in tiled matrix multiplication kernels that use shared memory?

A. To ensure all threads have finished loading a tile into shared memory before any thread starts using that data

B. To synchronize the GPU with an external network device

C. To convert shared memory into global memory

D. To permanently disable shared memory after use

Correct Answer: A

Explanation: Without this synchronization barrier, some threads might start computing with tile data before other threads have finished loading it, leading to incorrect results.

51. What is register spilling in the context of CUDA kernel performance?

A. When shared memory usage exceeds the block's allotted size only

B. When threads run out of available blocks to be assigned to

C. When a kernel uses more registers per thread than are available, forcing some variables into slower local/global memory

D. When a GPU physically overheats and shuts down

Correct Answer: C

Explanation: Register spilling happens when a kernel's register usage per thread exceeds the hardware limit, causing the compiler to place some variables in slower memory instead.

52. Why might reducing a kernel's register usage sometimes improve overall performance?

A. Registers have no effect on how many threads can run concurrently

B. Reducing registers always slows down every kernel

C. Lower register usage per thread can allow more threads or blocks to be resident simultaneously on an SM, improving occupancy

D. Registers are unrelated to GPU thread scheduling

Correct Answer: C

Explanation: Since SM resources like registers are shared among resident threads, using fewer per thread can allow more threads to run concurrently, which may raise occupancy.

53. What is the difference between global memory and shared memory in terms of scope on a CUDA GPU?

A. Global memory is accessible by all threads across the entire grid, while shared memory is only accessible within a single block

B. Shared memory can be accessed from the host CPU directly

C. There is no difference between the two

D. Global memory can only be accessed by one single thread

Correct Answer: A

Explanation: Global memory is visible to every thread in the kernel launch, whereas shared memory is scoped only to the threads within the same block.

54. What is a common use case for using cuFFT or similar specialized NVIDIA libraries alongside custom CUDA kernels?

A. To leverage highly optimized implementations of common operations, like Fast Fourier Transforms, instead of writing them from scratch

B. To convert CUDA code into CPU-only code

C. To manage GPU driver updates automatically

D. To replace the need for any custom CUDA code

Correct Answer: A

Explanation: Specialized libraries like cuFFT provide vendor-tuned implementations of common but complex operations, saving development time and often outperforming hand-written kernels.

55. Why might a CUDA programmer benchmark a kernel using GPU profiling tools rather than relying solely on theoretical analysis?

A. Theoretical analysis alone always perfectly predicts real-world performance

B. Actual performance can be affected by real hardware behavior, such as memory access patterns and occupancy, that are hard to predict purely in theory

C. Profiling is only useful for CPU code, not GPU kernels

D. Profiling tools always give incorrect results compared to theory

Correct Answer: B

Explanation: Because real GPU performance depends on many interacting hardware factors, empirical profiling often reveals bottlenecks that pure theoretical analysis might miss.

56. What is a key reason CUDA programming knowledge is valuable for AI and deep learning practitioners specifically?

A. Deep learning only runs efficiently on CPUs

B. CUDA is unrelated to how deep learning frameworks execute computations

C. AI workloads never use GPUs in practice

D. Deep learning training and inference rely heavily on GPU-accelerated matrix and tensor operations

Correct Answer: D

Explanation: Because deep learning workloads are dominated by large matrix and tensor computations, understanding CUDA and GPU acceleration is directly relevant to optimizing AI performance.

57. Why might a course cover both low-level shared memory/atomic operations and high-level libraries like cuBLAS or CuPy?

A. Because low-level and high-level approaches are functionally identical

B. To give learners both a deep understanding of GPU internals and practical tools for everyday productivity

C. Because only one of the two approaches is ever used professionally

D. Because high-level libraries make learning low-level concepts unnecessary in all cases

Correct Answer: B

Explanation: Covering both equips learners to understand what's happening under the hood while also being productive with optimized, ready-made libraries in real projects.

58. What is a good general strategy when first approaching a new, unfamiliar GPU programming problem?

A. Start with a straightforward, correct implementation, then profile and optimize based on identified bottlenecks

B. Immediately attempt the most complex optimization technique available

C. Skip correctness testing and focus only on speed

D. Avoid using any profiling tools throughout development

Correct Answer: A

Explanation: A common and effective approach is to first get a correct baseline implementation working, then use profiling to identify and address the actual performance bottlenecks.

59. What is an overarching theme connecting topics like Tensor Cores, memory coalescing, shared memory, and occupancy across a CUDA programming course?

A. Avoiding the use of any specialized GPU hardware features

B. Maximizing effective utilization of the GPU's parallel hardware resources for a given workload

C. Ensuring code only runs correctly on a single specific GPU model

D. Minimizing the total number of lines of code written

Correct Answer: B

Explanation: Each of these topics, in its own way, is about better utilizing the GPU's parallel compute and memory resources to achieve higher performance.

60. Why is double-checking kernel correctness on small test cases important before scaling up to large matrices or datasets on the GPU?

A. GPUs cannot process large datasets under any circumstances

B. Bugs are often easier to identify and debug on small, verifiable inputs before dealing with large-scale performance issues

C. Small test cases always guarantee a kernel is fully optimized

D. Correctness testing is unnecessary once a kernel compiles without errors

Correct Answer: B

Explanation: Verifying correctness on small, easily checked inputs first helps catch logic errors early, before they become harder to diagnose at scale.